

Création et destruction d'un processus

Création de processus

- Créer un processus, consiste à lui attribuer un identificateur et initialiser son état.
- L'identificateur doit être unique pour distinguer sans ambiguïté le processus.
- Un processus qui vient d'être créé peut à son tour donner naissance à d'autres processus. Ce processus est appelé processus père et les descendants constituent des fils.

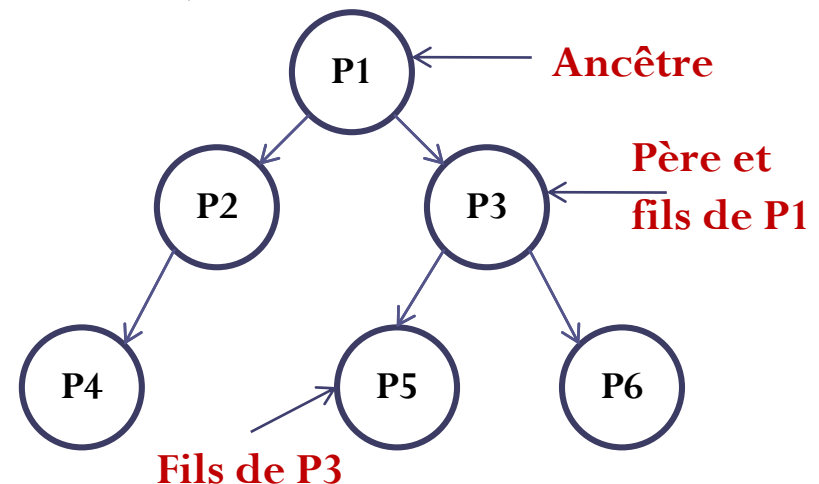


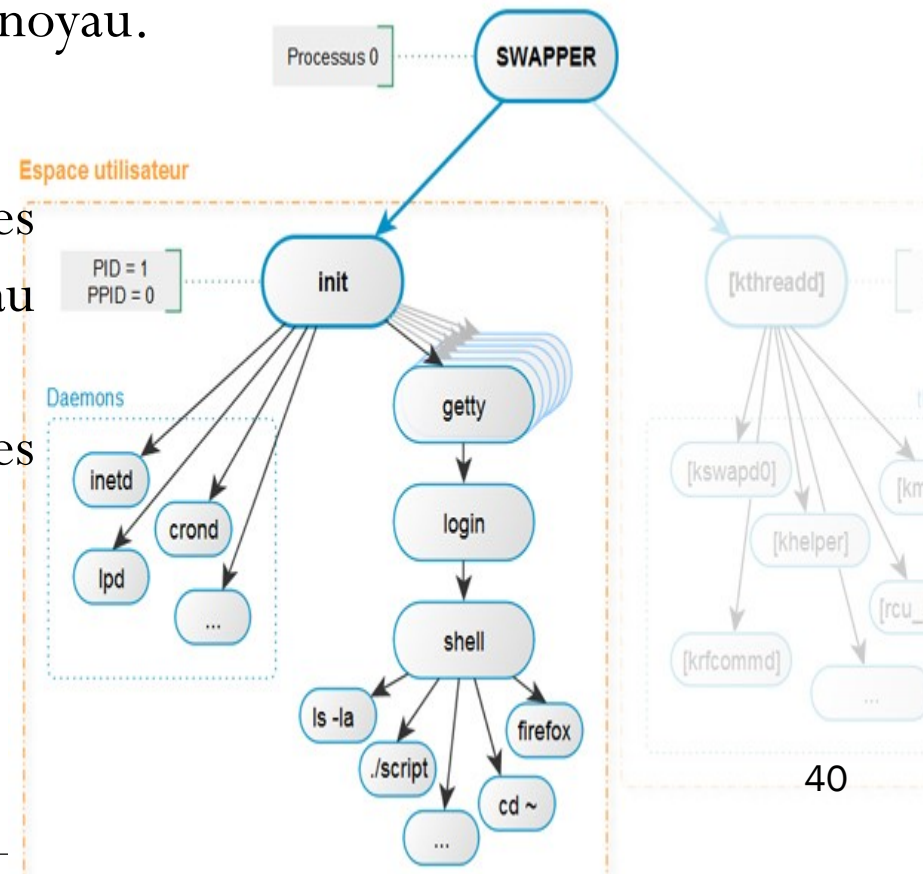
Figure1: L'arbre des processus 39

Création de processus

Lors du démarrage de Unix/Linux, 3 processus sont créés :

1. le Swapper (pid = 0) qui gère la mémoire et crée les 2 autres processus;
2. le Init (pid = 1, ppid=0) qui crée tous les autres processus dans l'espace utilisateur.
3. Kthreadd (pid = 2, ppid=0) à l'espace noyau.

- **Init** se chargera ensuite de créer les autres processus nécessaires au fonctionnement du système.
- **Init** sera ici l'ancêtre de tous les processus.



Destruction de processus

- La destruction d'un processus peut avoir lieu pour deux raisons :
 - Destruction normale (ordinaire) : le processus vient de se terminer d'une façon normale car il a achevé le traitement pour lequel il a été créé.
 - Destruction anormale : le système d'exploitation met fin à un processus sans qu'il termine correctement son objectif car il vient de causer un mauvais fonctionnement.
- La destruction d'un processus conduit à la suppression de ses propriétés et la libération de toutes les ressources qui ont été occupées.

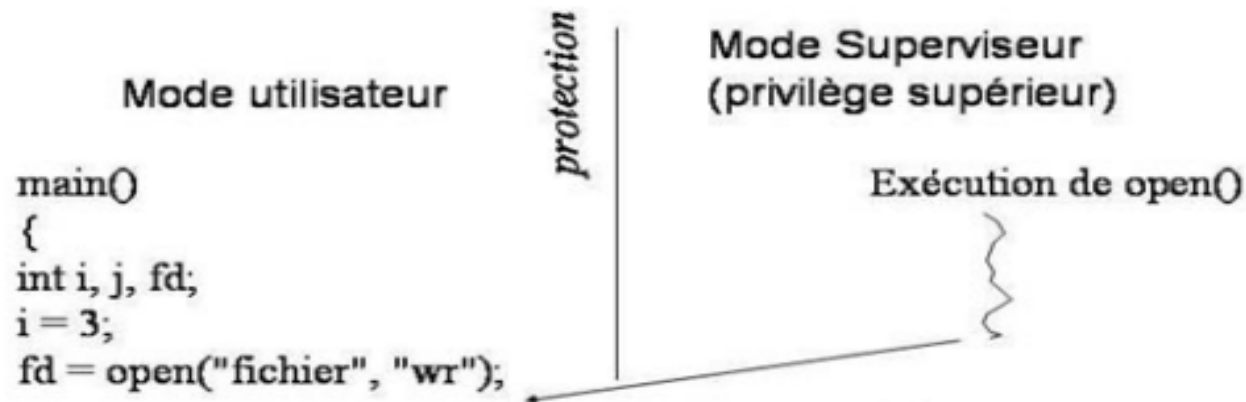
Appels système (1/2)

- **Appel système** est une demande de service c'est-à-dire lorsqu'un processus a besoin d'exécuter une opération spécifique, il demande au SE de la réaliser pour son compte et lui envoyer le résultat
- Lorsqu'une application souhaite accéder à une ressource, ou plus généralement invoquer une fonction du SE, elle procède à un **appel système**.
- Les appels systèmes se présentent donc comme des fonctions/procédures classiques.
 - **open, read, write** et **close** qui permettent les manipulations sur les systèmes de fichiers

Appels système (2/2)

- Exemple:

```
fd = open("mon_fichier.txt", O_WRONLY);  
write(fd, "hello world\n", 12);  
close(fd);
```



Création de processus avec `fork()` sous UNIX

- L'appel système *fork()* est une façon qui permet de créer des processus aussi appelés **processus lourds** :
 - Création par des appels systèmes lents
 - Changement de contexte lent
 - Espace d'adressage propre (pas de partage de la mémoire simple)
 - Copies de toutes les variables et ressources nécessaires (pile, registres, fichiers ouverts...)
 - Mécanismes de communication lourds (pipe, signaux...)
 - ...

Exemple 1 : Instruction de création :

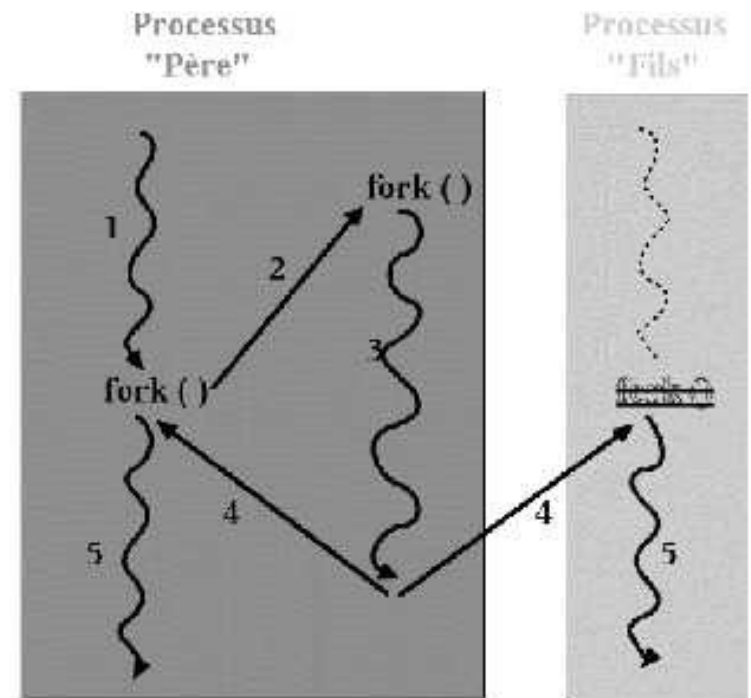
```
#include <unistd.h>
```

```
int fork();
```

- `fork()` est le moyen de créer des processus, par duplication d'un processus existant.
- L'appel système `fork()` crée une copie exacte du processus original ainsi que son contexte initial.

Création de processus avec fork() sous UNIX

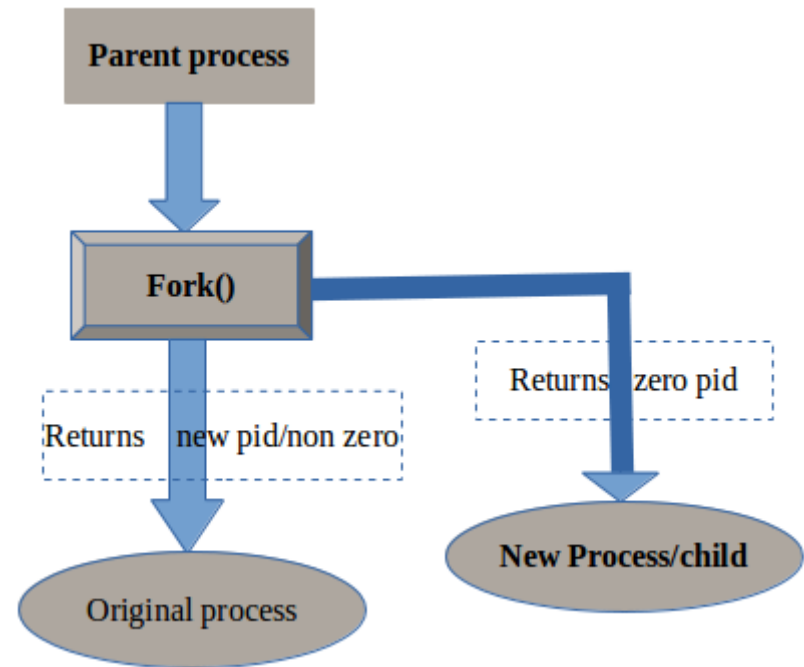
- Les deux processus sont exactement identiques et exécutent le même programme sur **deux copies distinctes** des données. Les espaces de données sont complètement séparés : la modification d'une variable dans l'un est invisible dans l'autre.
- Les processus "père" et "fils" deviennent concurrents (phase n°5) vis-à-vis de l'ordonnanceur. Le processus ainsi créé hérite, du processus qui l'a créé, un certain nombre d'attributs :
 - Le même code,
 - Une copie de la zone de données,
 - L'environnement,
 - La priorité,
 - Les descripteurs de fichiers courants (mêmes caches et mêmes pointeurs),
 - Le traitement des signaux.



Création de processus avec fork() sous UNIX

Pour distinguer le processus père du processus fils on regarde la valeur de retour de fork(), qui peut être :

- La valeur 0 dans le processus fils.
- Positive pour le processus père et qui correspond au PID du processus fils
- Négative si la création de processus a échoué ;



Création de processus avec `fork()` sous UNIX

La fonction **`fork()`** réalise les tâches suivantes :

- 1) Test d'existence de ressources mémoires suffisantes pour permettre la création du nouveau processus,
- 2) Calcul du PID du processus enfant. Le noyau explore ensuite la table `proc[]` pour vérifier que ce numéro n'est pas en cours d'utilisation. Sinon le calcul est relancé,
- 3) Recherche d'une entrée libre dans la table des processus,
- 4) Duplication du contexte processeur et mémoire du processus parent,
- 5) Mise à jour des tables système modifiées par la création du nouveau processus.

Création de processus avec fork() sous UNIX

Puisque les processus créés par fork() s'exécutent de façon concurrente avec le père, on ne peut pas prévoir la politique et l'état de l'ordonnanceur du système,

- Il sera impossible de déterminer quels processus se termineront avant tels autres (y compris leur père).
- Si le père est indisponible à la terminaison de son fils, on dit que le fils est dans un état zombi.

Zombi est un processus qui s'est achevé, mais qui dispose toujours d'un identifiant de processus (PID) et reste donc encore visible dans la table des processus.

- Si un processus père termine avant son fils, le noyau rattache le processus fils (**orphelin**) au processus **init**, d'où l'existence, dans certains cas, d'un problème de synchronisation

Application

Préciser le nombre de processus créés par les programmes ci-dessous puis dessiner l'arbre généalogique des processus engendrés et indiquer les valeurs de variables pour chaque processus :

Code 1	Code 2	Code 3	Code 4	Code 5
<pre>int main() { fork(); fork(); fork(); }</pre>	<pre>int main() { int x,y,z; x=fork(); y=fork(); z=fork(); }</pre>	<pre>int main() { int x=0; int y=0; if (fork() > 0) x=2; else y=4; }</pre>	<pre>int main() { if (fork() > 0) fork(); }</pre>	<pre>int main() { int cpt=0; while (cpt < 3) { if (fork() > 0) cpt++; else cpt=3; }}</pre>